Software Configuration Management Plan (SCMP)

Project: GitScout – GitHub Talent Intelligence Platform

Version: 1.0
Date: February 3, 2026

1. Purpose and Scope

This document combines the Software Configuration Management Plan (SCMP) and the Software Project Management Plan (SPMP) for GitScout. It defines:

- Project organization and responsibilities
- Configuration management and version control procedures
- Project management workflow using GitHub Issues and GitHub Projects
- Risk management approach and risk register
- Estimation, scheduling, and tracking mechanisms
- Documentation standards and monitoring/observability plan

GitScout is a web-based recruiting and talent sourcing platform that analyzes public GitHub signals (e.g., activity and contribution metrics) to help identify and evaluate developer candidates. The plan applies to the MVP scope as described in the project proposal and supports iterative enhancements after MVP completion.

2. Organization

2.1 Team Roles

The project team is organized as follows:

Role	Owner(s)	Primary Responsibilities
Project Manager	Xiangan He	Scope control, milestones/schedule, risk review cadence, stakeholder coordination, release readiness sign-off
Frontend Developer	Khushi Sridhar; Xiangan He	Next.js UI, search/filter UX, dashboards, shortlist workflows, exports UI, responsive implementation

Backend Developer	Bowang Lang; Cong Chen; Xiangan He	API design (Elysia), GitHub API integration, scoring service, caching, database integration, security controls
UX/UI Designer	Haomiao Hu	User flows, wireframes, design system usage (Tailwind/shadcn), accessibility and usability passes
QA/Testing Lead	Xiangan He; Joseph Ma	Test strategy, CI quality gates, regression testing, release checklist, defect triage

2.2 Working Agreements

Cadence

- Standup: 2 times per week (15 minutes)
- Sprint planning: weekly (45-60 minutes)
- Demo + retrospective: weekly (45 minutes)

Definition of Done

- Acceptance criteria met and validated
- Tests added/updated and passing in CI
- Documentation updated (README, ADRs, API docs where relevant)
- Code reviewed and merged via pull request

2.3 Decision-Making

Decisions are made using a lightweight, documented approach:

- Product scope and schedule tradeoffs are owned by the Project Manager.
- Technical design decisions are owned by the relevant area owner (frontend/back-end), documented as ADRs when impactful.
- UX/UI decisions are owned by the UX/UI Designer in consultation with the frontend team.

3. SCMP: Configuration Management

3.1 Configuration Identification:

Configuration identification defines the project artifacts placed under version control and establishes naming, ownership, and lifecycle tracking for each configuration item:

- Source code (web, server, shared packages)
- Database schema and migrations

- Infrastructure/deployment configuration (Railway settings, environment variables templates, secrets references)
- Build/CI configuration and scripts
- Documentation (README, ADRs, API docs, runbooks)
- Test assets and fixtures

3.2 Repository Structure

Recommended monorepo structure:

- /web - Next.js frontend
- /server - Elysia (Bun) backend
- /packages/* - shared TypeScript utilities, types, GitHub client wrappers, scoring logic
- /docs - ADRs, runbooks, architecture notes

3.3 Configuration Control

Configuration control is implemented using GitHub to manage review, approval, and traceability of changes to configuration items. The branching strategy supports controlled change integration and maintains configuration integrity:

- Short-lived feature branches off main (e.g., feature/search-filters, fix/export-bug).
- Pull requests are required for all merges into main to enforce review and approval.
- Trunk-based habits: small PRs, frequent merges, avoid long-running divergence.

3.4 Configuration Status Accounting

Configuration status accounting activities will be maintained through GitHub Issues, pull request history, and tagged releases. The team will track feature completion, defect resolution, and baseline versions to maintain visibility into project evolution and ensure traceability of configuration items and their changes.

3.5 Configuration Evaluation and Reviews

The team will conduct periodic reviews of configuration items during sprint retrospectives. Pull request reviews, milestone demos, and release checkpoints act as evaluation points to ensure alignment with project requirements and approved baselines.

3.6 Pull Request Policy

All changes are integrated via pull requests with the following minimum requirements:

- Link to relevant GitHub issue(s)
- Clear description of what/why
- UI changes include screenshots or a short video clip
- Tests updated or added (or a written justification if not applicable)
- CI checks must pass and at least one reviewer approval

3.7 Baselines, Releases, and Change Control

Baselines are defined per milestone (M0, M1, M2, M3, M4). For each baseline:

- Tag the repository with a semantic version tag (e.g., v0.1.0-m1).

- Record a short changelog in GitHub Releases.
- Capture deployment notes (env var changes, migrations) in /docs/runbooks.

Scope changes are controlled through milestone planning. New features must be added to the backlog and explicitly scheduled; they should not be introduced ad hoc late in a sprint unless they are severity-1 defects.

3.8 SCM Plan Maintenance

This SCMP is treated as a living document and may be updated through pull requests as project practices evolve.

4. Project Management Tooling (GitHub Issues / Projects)

4.1 GitHub Issues

GitHub Issues are used for all work tracking. Each issue should include:

- Problem statement and context
- Acceptance criteria (testable checklist)
- Dependencies (if any)
- Test plan
- Definition of done checklist (optional but encouraged)

Recommended labels:

- area:web, area:server, area:db, area:ux, area:qa, area:devops
- type:feature, type:bug, type:chore, type:spike
- priority:p0/p1/p2
- size:xs/s/m/l
- risk:api, risk:privacy, risk:performance

4.2 GitHub Projects Board

A single GitHub Project board is recommended with the following columns:

- Backlog
- Ready
- In progress
- In review
- Done

Views should include Sprint view, Risk view (filter by risk:* labels), and Release view (by milestone).

5. Risk Management Plans

5.1 Risk Process

Risk management follows an iterative review model aligned with sprint plannings:

- Risks are tracked as issues labeled risk:* and reviewed weekly during planning.
- Each risk must have an owner, mitigation steps, and a validation plan.
- If a risk results in an incident, create a postmortem issue with concrete action items.

5.2 Initial Risk Register

ID	Risk	Likelihood	Impact	Mitigations / Owner
R1	GitHub API rate limits or quota constraints	High	High	Use GraphQL where beneficial; cache responses with TTL; exponential backoff; partial-results UI; monitor calls per search. Owner: Backend
R2	Scoring produces misleading outcomes or is hard to explain	Medium	High	Transparent scoring explanation in UI; version the scoring algorithm; log score inputs; validate missing-data cases. Owner: PM + Backend
R3	Privacy/compliance concerns from storing candidate data	Medium	High	Only use public GitHub data; minimize PII; document data stored; support deletion of shortlists; add limitations page. Owner: PM

R4	Scope creep from post-MVP features	High	Medium	Milestone gates; maintain explicit out-of-scope list; only swap scope with PM approval. Owner: PM
R5	Performance issues (search latency, dashboard rendering)	Medium	Medium/High	Pagination; database indexes; caching; async aggregation for heavy profiles if needed; define perf budgets and track p95 latency. Owner: Backend + Frontend

6. Estimation and Scheduling

6.1 Estimation Method

The team uses relative estimation (XS/S/M/L) and plans work in short iterations. Time estimates are derived only at sprint planning and validated against delivered work to improve accuracy.

6.2 Milestones and High-Level Schedule

A suggested MVP plan (adjust to course timeline):

- M0 - Project Setup: Repo/CI setup; base Next.js UI shell; Elysia API skeleton + Swagger; database schema v0
- M1 - Search + Ingestion: GitHub API integration; advanced search + filtering; developer profile view
- M2 - Analytics + Scoring: Dashboards and contribution analytics; scoring algorithm v1; scoring explanation page
- M3 - Shortlists + Export + UX polish: Shortlisting workflow; export to CSV/PDF; responsive UX pass
- M4 - Stabilization + Release: Test hardening; performance improvements; deployment and monitoring baseline; final demo

6.3 Tracking and Control

Tracking is performed via GitHub Projects and weekly reviews. Change control is milestone-based: new features enter the backlog and are scheduled explicitly; urgent changes during a sprint are limited to high-severity defects.

7. Documentation, Reporting and Monitoring

7.1 Documentation Standards

Documentation is maintained in-repo and kept current with implementation:

- README.md - setup, development, testing, deployment
- CONTRIBUTING.md - branching/PR rules, coding standards
- docs/adr/ - Architecture Decision Records for significant technical decisions
- docs/api/ - API contract, examples, and Swagger reference
- docs/runbooks/ - operational runbooks (rate limits, migrations, rollbacks)
- User-facing docs - scoring explanation, data sources/limitations

7.2 Monitoring and Observability (MVP)

The MVP monitoring plan emphasizes practical signals that catch failures quickly:

- Structured server logs with request id, route, status code, latency, GitHub API calls per request
- Health endpoint (/healthz) and uptime checks
- Error tracking via log aggregation and alerting on spikes
- Database monitoring: slow query sampling and index checks for search fields

7.3 Key Metrics

Track the following metrics to understand stability and user experience:

- Search latency p50 and p95
- GitHub API requests per search and per profile load
- Cache hit rate
- Error rate by endpoint
- Export success rate (CSV/PDF)